Fisher 判别分析实验报告

反向链接 · 1　　更新时间 刚刚

信息 ▴

作业说明：

要求：在UCI数据集上的Iris和sonar数据上验证算法的有效性；Iris数据3类，4维，150个数据；Sonar数据2类，60维，208个样本；

训练和测试样本有三种方式进行划分：（三选一）

1）将数据随机分训练和测试，多次平均求结果

2）k折交叉验证

3）留1法

（针对不同维数，画出曲线图；）

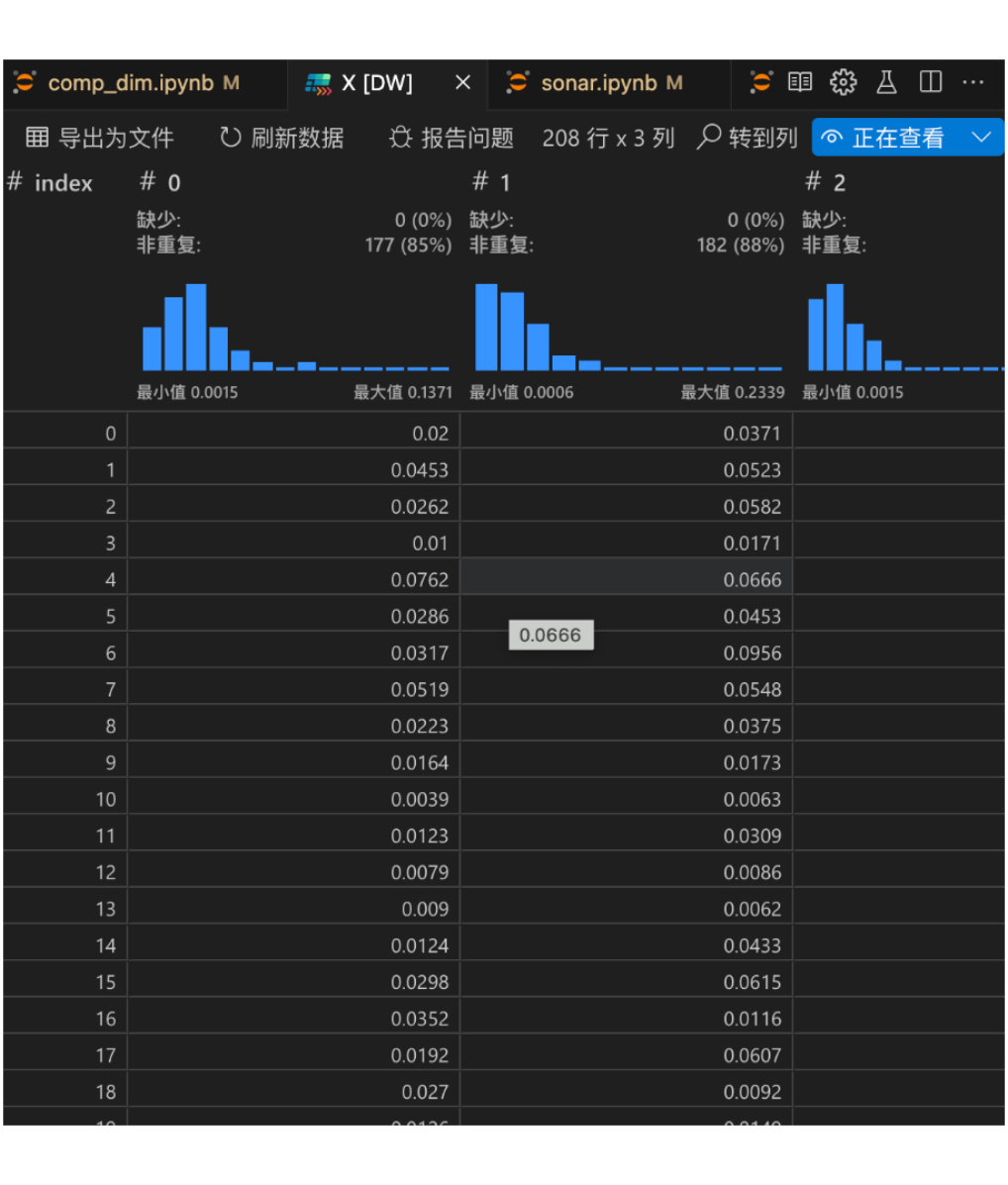
仿真结果+报告。

数据地址：http://archive.ics.uci.edu/ml/index.php

操作系统：macos

python_version==3.10

tools：vscode插件-Data Wrangler 用于可视化 dataframe、numpy 数据



# index	# 0	# 1	# 2
	缺少: 0 (0%) 非重复: 177 (85%)	缺少: 0 (0%) 非重复: 182 (88%)	缺少: 非重复:
0	0.02	0.0371	
1	0.0453	0.0523	
2	0.0262	0.0582	
3	0.01	0.0171	
4	0.0762	0.0666	
5	0.0286	0.0453	
6	0.0317	0.0956	
7	0.0519	0.0548	
8	0.0223	0.0375	
9	0.0164	0.0173	
10	0.0039	0.0063	
11	0.0123	0.0309	
12	0.0079	0.0086	
13	0.009	0.0062	
14	0.0124	0.0433	
15	0.0298	0.0615	
16	0.0352	0.0116	
17	0.0192	0.0607	
18	0.027	0.0092	

开源仓库：https://github.com/little1d/pattern_recognition

数据集介绍

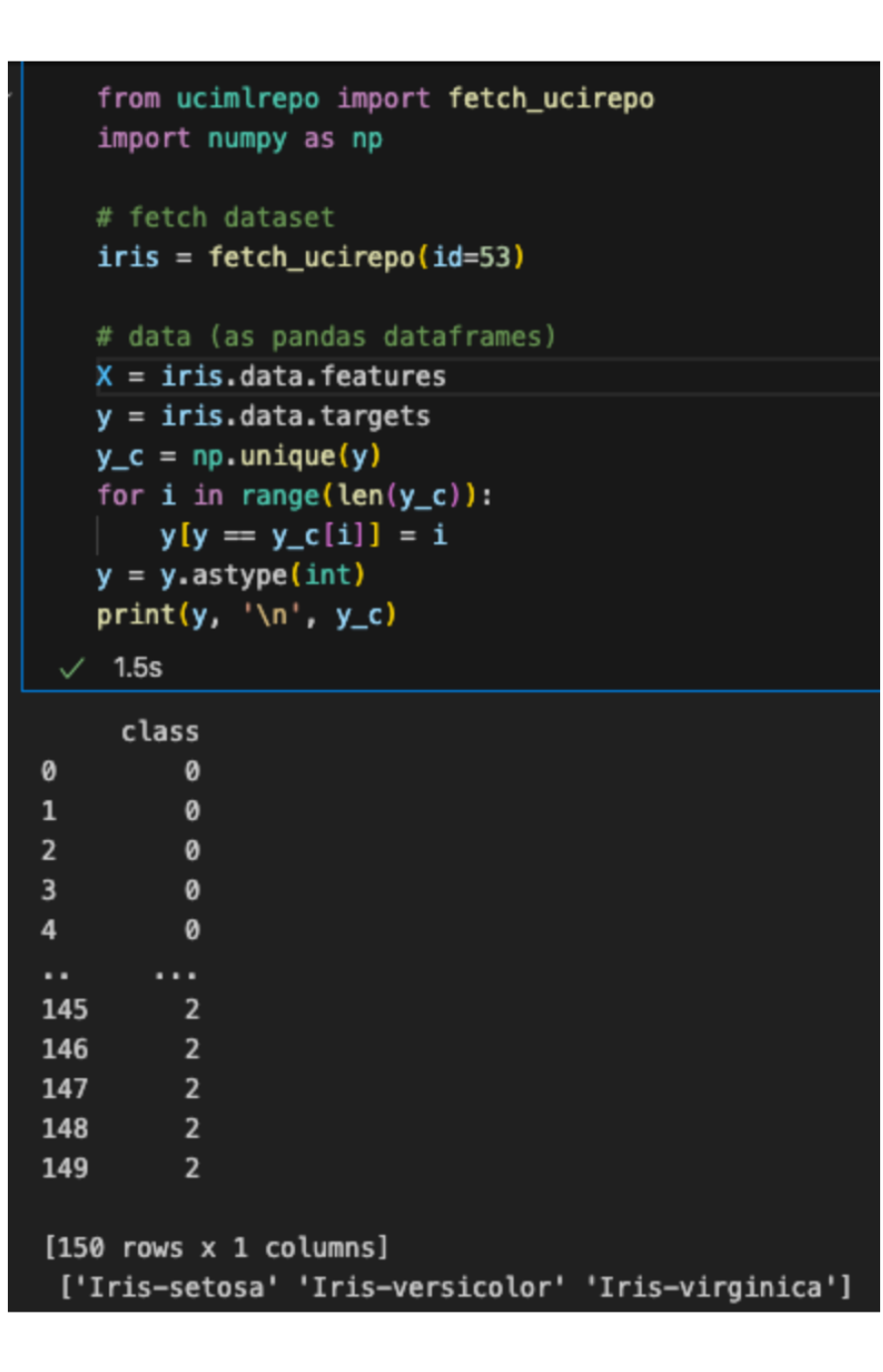
1. Sonar

   Sonar 数据集是一个经典的二分类问题数据集，源于对声呐的分析，目的是通过声纳回波来区分海底的岩石（rock）和水雷（mine），60 维表示有 60 个数值型特征

2. Iris

   它包含了 150 个样本，每个样本都有四个特征：花萼（Sepal）的长度和宽度，以及花瓣（Petal）的长度和宽度。这些样本分别来自三种不同的鸢尾花品种：Setosa、Versicolor 和 Virginica。

数据集加载及划分方法



```
from ucimlrepo import fetch_ucirepo
import numpy as np

# fetch dataset
iris = fetch_ucirepo(id=53)

# data (as pandas dataframes)
X = iris.data.features
y = iris.data.targets
y_c = np.unique(y)
for i in range(len(y_c)):
    y[y == y_c[i]] = i
y = y.astype(int)
print(y, '\n', y_c)
```

```
✓ 1.5s
     class
0        0
1        0
2        0
3        0
4        0
..     ...
145      2
146      2
147      2
148      2
149      2

[150 rows x 1 columns]
 ['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

同时应该对标签进行处理。iris 数据集的标签是Setosa、Versicolor 和 Virginica。而sonar 数据集是“R”或“M”。我们可以对应编码成 1,2,3。方便统一读取设计。

使用 np.unique(y)，获取 y 的类别，再在数据集中进行替换。

最后再加入一些异常处理机制，最后封装成这样一个函数

```
def load_data(dataset):
    try:
        if dataset == 'sonar':
            data = fetch_ucirepo(id=151)
        elif dataset == 'iris':
            data = fetch_ucirepo(id=53)
    except Exception as e:
        raise ValueError(f"Error occurred while
    # 将 dataframes 转为 numpy arrays
     X = data.data.features.to_numpy()
     y = data.data.targets.to_numpy()
     y_c = np.unique(y)
     for i in range(len(y_c)):
         y[y == y_c[i]] = i
     y = y.astype(int)
     return X, y
```

小 tips：

尽量将 dataframe 转换为 ndarray，转换方法有df.values,df.to_numpy

dataframe 属于 pandas，需要通过行列标签来访问元素，当已知信息为索引时，使用 iloc 函数。这样可以避免频繁的转换。

获取到原始数据集后，我们需要对训练集数据集划分，提供了三种方法：

1. 将数据随机分类
2. k 折验证
3. 留一法

针对这些方法，还有一些问题：

1. 无论是训练集还是测试集，要保证每个类别均有样本，且样本比例应该相等。
2. 而且获取到的数据集类别是按顺序排列的，需要随机打乱。

我这里参考的是 scikit-learn 库的三种数据集划分方法，参考链接如下。

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection

针对问题一，随机分类函数 train_test_split 提供参数stratify。用于将数据按类别随机划分。

> **stratify** : ***array-like, default=None***
>
> If not None, data is split in a stratified fashion, using this as the class labels.

kfold 方法，sklearn 提供了 StratifiedKFold 方法保证这点。

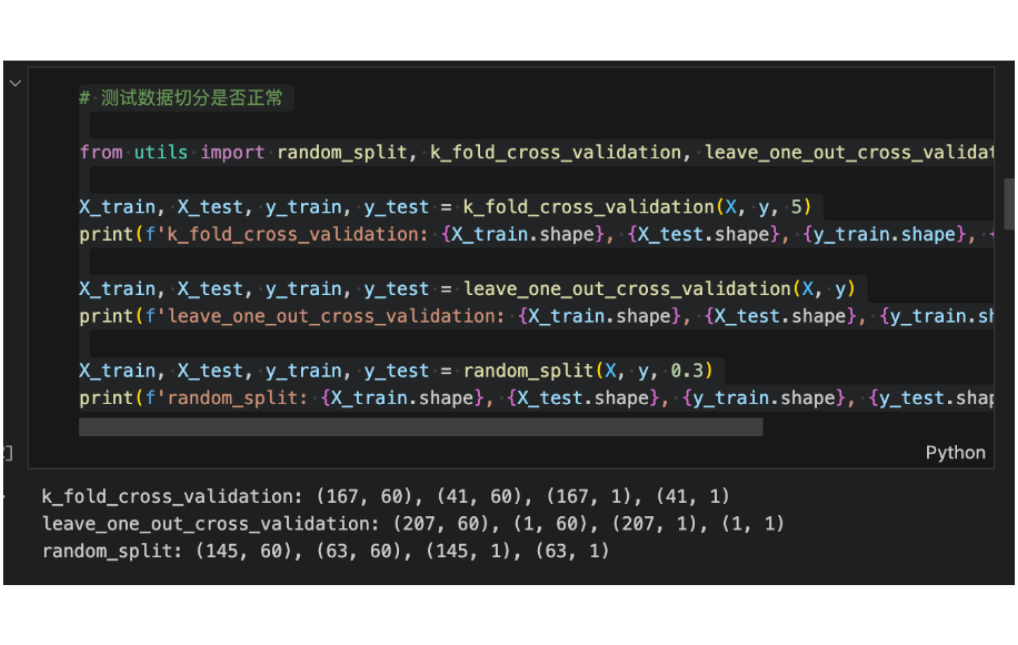
针对问题二，这两个方法都提供了 shuffle 参数，用于提前将数据打乱。将 shuffle 初始化为 True

> **shuffle** : ***bool, default=False***
>
> Whether to shuffle each class's samples before splitting into batches. Note that the samples within each split will not be shuffled.

最后封装成参数放在 utils.py 中

```
def random_split(X,y,test_size=0.3,random_state
    X_train, X_test, y_train, y_test = train_te
    return X_train, X_test, y_train, y_test

def k_fold_cross_validation(X, y, n_splits, ran
    skf = StratifiedKFold(n_splits=n_splits,ran
    skf.get_n_splits(X,y)
    for _, (train_index, test_index) in enumera
        X_train, X_test = X[train_index], X[tes
         y_train, y_test = y[train_index], y[te
     return X_train, X_test, y_train, y_test

 def leave_one_out_cross_validation(X,y):
     # dataframes to numpy arrays
     loo = LeaveOneOut()
     loo.get_n_splits(X)
     for _, (train_index, test_index) in enumer
         X_train, X_test = X[train_index], X[te
         y_train, y_test = y[train_index], y[te
     return X_train, X_test, y_train, y_test
```

对函数进行测试，打印划分好的数据集形状



```
# 测试数据切分是否正常

from utils import random_split, k_fold_cross_validation, leave_one_out_cross_validat

X_train, X_test, y_train, y_test = k_fold_cross_validation(X, y, 5)
print(f'k_fold_cross_validation: {X_train.shape}, {X_test.shape}, {y_train.shape},

X_train, X_test, y_train, y_test = leave_one_out_cross_validation(X, y)
print(f'leave_one_out_cross_validation: {X_train.shape}, {X_test.shape}, {y_train.sh

X_train, X_test, y_train, y_test = random_split(X, y, 0.3)
print(f'random_split: {X_train.shape}, {X_test.shape}, {y_train.shape}, {y_test.sha
```

```
k_fold_cross_validation: (167, 60), (41, 60), (167, 1), (41, 1)
leave_one_out_cross_validation: (207, 60), (1, 60), (207, 1), (1, 1)
random_split: (145, 60), (63, 60), (145, 1), (63, 1)
```

Fisher 判别分析

Fisher 判别分析是一种有监督学习的降维技术，但是其只能完成二分类任务。iris 共有三类，我们可以将其两两成组比较，这样就有三个二分类实验了。

根据 Fisher 判别分析的定义，我们先要根据样本求解类内离散矩阵的逆，最后求得 W 作为分类向量。

先定义一个子函数，用于根据一组样本求解其均值和特征的协方差矩阵。

```
def cal_cov_and_avg(samples):
    # samples size m*n m 为样本数, n 为特征维数
    mu = np.mean(samples, axis=0)
    # 协方差矩阵shape 为 n*n, n 为特征维数, 需要先转
    cov_m = np.cov(samples.T)
    return cov_m, mu
```

假设 m 为样本数，n 表示特征数。这里的 samples 就是整个样本，设置 axis=0 意为对每一列求均值，即最后得到一个 1*n 的矩阵 mu 作为均值。

求解协方差矩阵时，我们需要将 samples 进行转置，求解的 con_m 形状为 n*n。

再定义 fisher 判别主函数，根据不同类别求解协方差矩阵，均值，再求出 W。因为后面需要判别某个样本属于哪一类需要均值，而且我们已经在此函数中求解过了，所以不妨输出出去。

```
# 定义 Fisher 判别分析函数
def fisher_discriminant_analysis(X, y):
    index1 = np.where(y == 0)[0] # 获取类别为'0'的
    index2 = np.where(y == 1)[0] # 获取类别为'1'的
    con_1, u1 = cal_cov_and_avg(X[index1])
    con_2, u2 = cal_cov_and_avg(X[index2])
    S_W = con_1 + con_2
    # 奇异值分解，提高数值稳定性
    u, s, v = np.linalg.svd(S_W)
     # 类内离散度矩阵的逆
     S_W_inv = np.dot(np.dot(v.T, np.linalg.inv
     # S_W_inv = np.linalg.inv(S_W)
     W = np.dot(S_W_inv,u1 - u2)
     return W,u1,u2
```

有效性验证

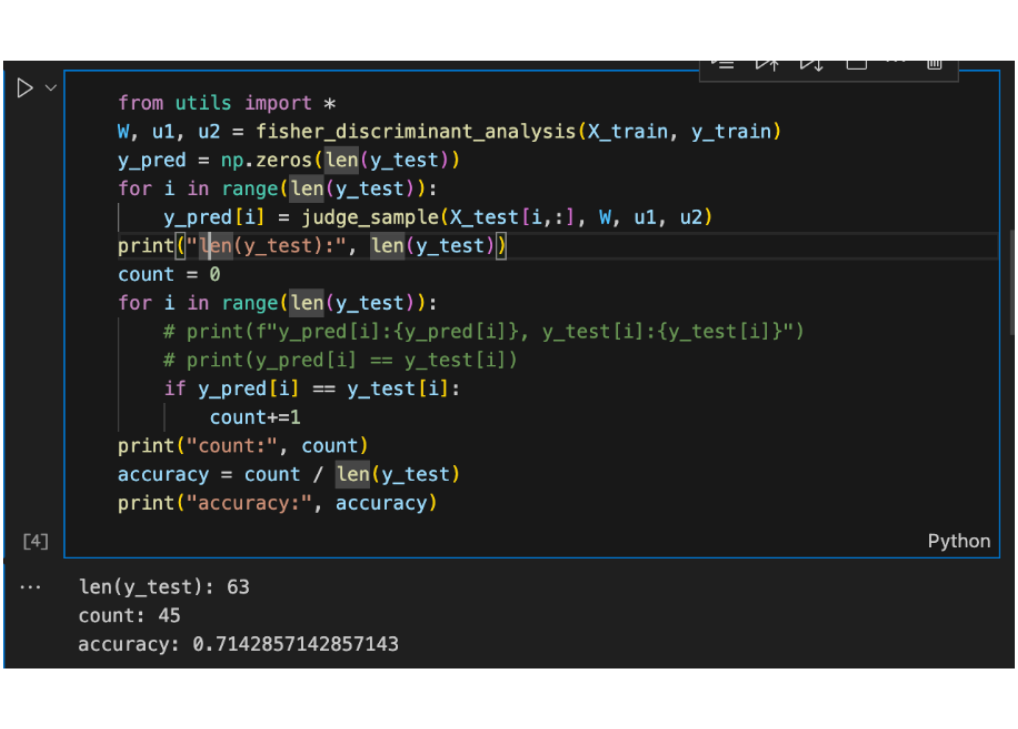
我们有了二分类的 w，现在需要根据测试样本的特征求解其距两类均值中心的距离，抽象成一个 judge_sample 函数

```
def judge_sample(sample, w, u1, u2):
    # sample 属于 c1 返回 False, 属于 c2 返回 True
    center_1 = np.dot(w.T, u1)
    center_2 = np.dot(w.T, u2)
    pos = np.dot(w.T, sample)
    return abs(pos - center_1) > abs(pos - cent
```

这个函数有个小细节，我并没有对距离再进行 if 比较，我直接返回了一个布尔值。当样本属于 class1，返回的就是 False，class1 中 y 的索引为 0，在 main 函数进行比较时可以直接判断 return 的值与 y_test。

算是一个小小的设计了，不过后面针对 iris 数据库的三类样本会遇到问题，后文会再说。

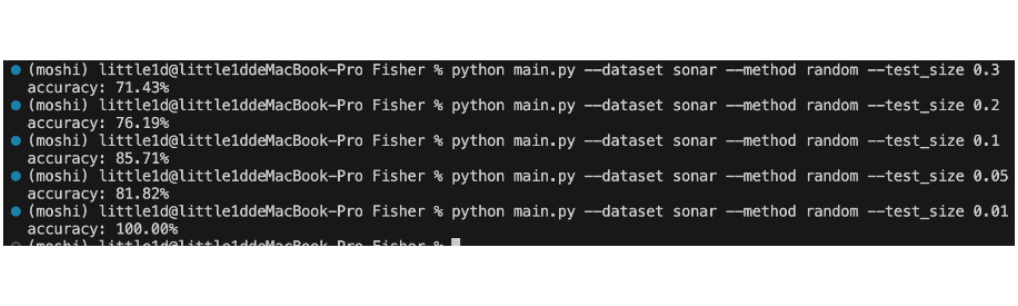
下面在 notebook 中对其简单测试



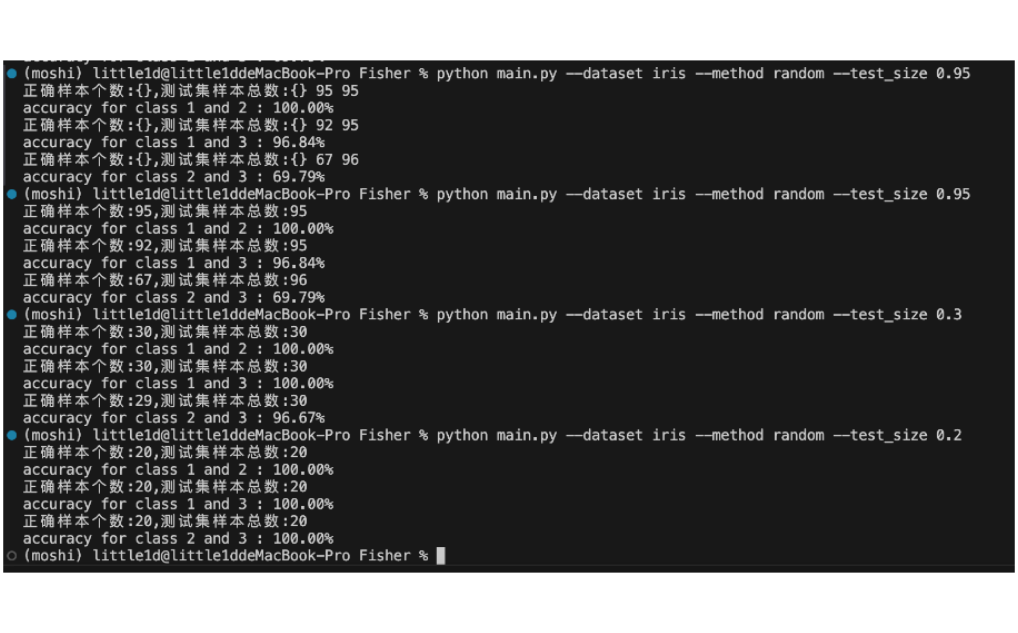
```
from utils import *
W, u1, u2 = fisher_discriminant_analysis(X_train, y_train)
y_pred = np.zeros(len(y_test))
for i in range(len(y_test)):
    y_pred[i] = judge_sample(X_test[i,:], W, u1, u2)
print("len(y_test):", len(y_test))
count = 0
for i in range(len(y_test)):
    # print(f"y_pred[i]:{y_pred[i]}, y_test[i]:{y_test[i]}")
    # print(y_pred[i] == y_test[i])
    if y_pred[i] == y_test[i]:
        count+=1
print("count:", count)
accuracy = count / len(y_test)
print("accuracy:", accuracy)
```

```
len(y_test): 63
count: 45
accuracy: 0.7142857142857143
```

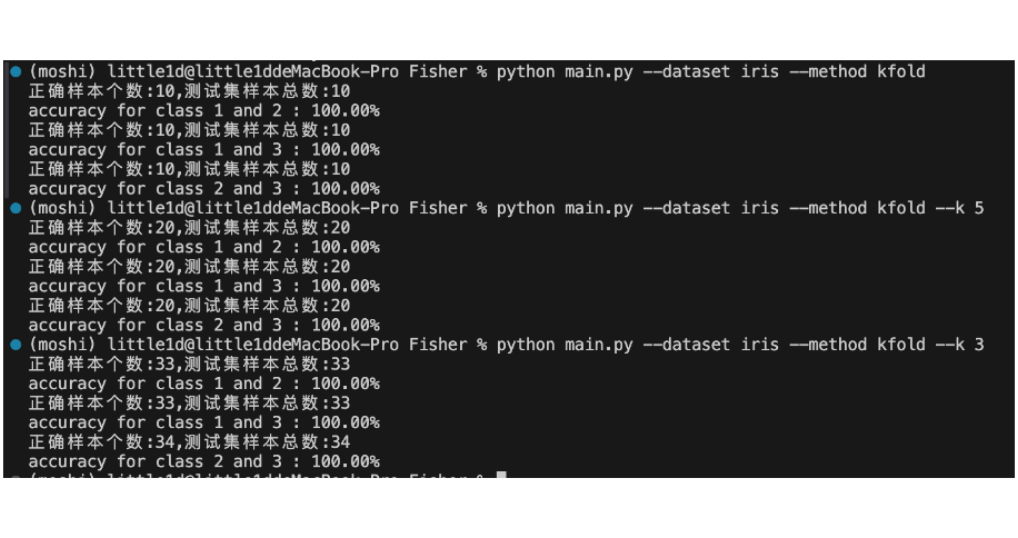
不断调整 test_size，改变训练集与数据集的比例，观察分类的正确率。可以看到当测试集比例越来越小时，分类的正确率越来越高。



```
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py —dataset sonar —method random —test_size 0.3
accuracy: 71.43%
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py —dataset sonar —method random —test_size 0.2
accuracy: 76.19%
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py —dataset sonar —method random —test_size 0.1
accuracy: 85.71%
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py —dataset sonar —method random —test_size 0.05
accuracy: 81.82%
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py —dataset sonar —method random —test_size 0.01
accuracy: 100.00%
```



```
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py —dataset iris —method random —test_size 0.95
正确样本个数:{},测试集样本总数:{} 95 95
accuracy for class 1 and 2 : 100.00%
正确样本个数:{},测试集样本总数:{} 92 95
accuracy for class 1 and 3 : 96.84%
正确样本个数:{},测试集样本总数:{} 67 96
accuracy for class 2 and 3 : 69.79%
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py —dataset iris —method random —test_size 0.95
正确样本个数:95,测试集样本总数:95
accuracy for class 1 and 2 : 100.00%
正确样本个数:92,测试集样本总数:95
accuracy for class 1 and 3 : 96.84%
正确样本个数:67,测试集样本总数:96
accuracy for class 2 and 3 : 69.79%
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py —dataset iris —method random —test_size 0.3
正确样本个数:30,测试集样本总数:30
accuracy for class 1 and 2 : 100.00%
正确样本个数:30,测试集样本总数:30
accuracy for class 1 and 3 : 100.00%
正确样本个数:29,测试集样本总数:30
accuracy for class 2 and 3 : 96.67%
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py —dataset iris —method random —test_size 0.2
正确样本个数:20,测试集样本总数:20
accuracy for class 1 and 2 : 100.00%
正确样本个数:20,测试集样本总数:20
accuracy for class 1 and 3 : 100.00%
正确样本个数:20,测试集样本总数:20
accuracy for class 2 and 3 : 100.00%
(moshi) little1d@little1ddeMacBook-Pro Fisher % ▌
```

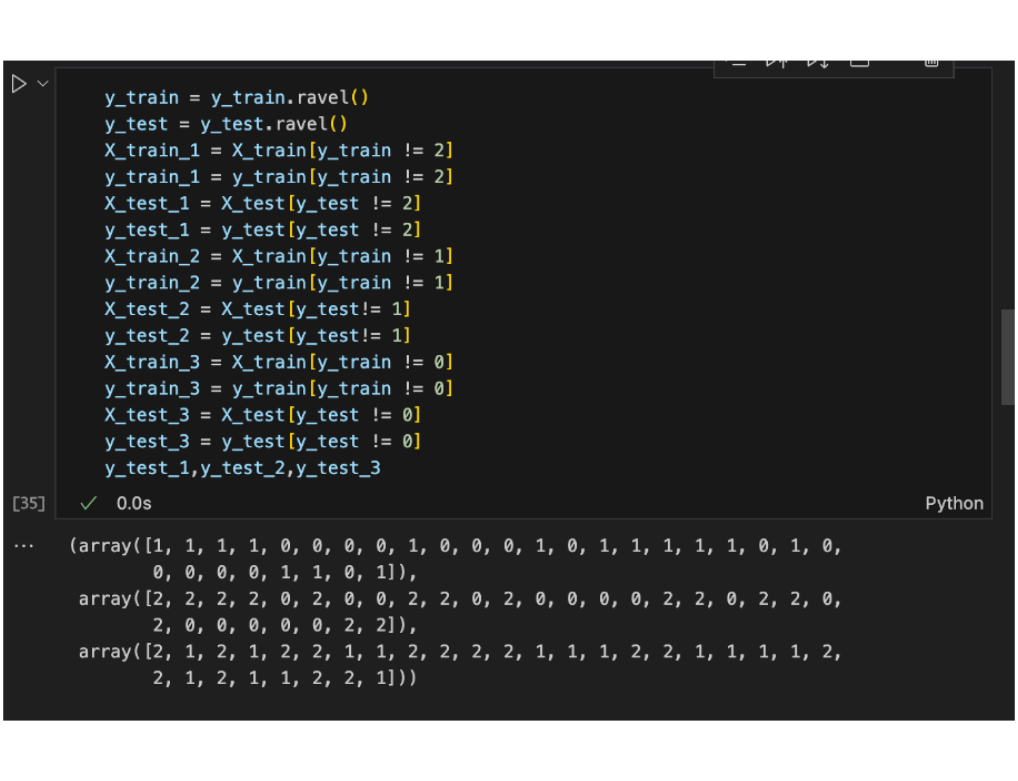


```
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py --dataset iris —method kfold
正确样本个数:10,测试集样本总数:10
accuracy for class 1 and 2 : 100.00%
正确样本个数:10,测试集样本总数:10
accuracy for class 1 and 3 : 100.00%
正确样本个数:10,测试集样本总数:10
accuracy for class 2 and 3 : 100.00%
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py --dataset iris —method kfold --k 5
正确样本个数:20,测试集样本总数:20
accuracy for class 1 and 2 : 100.00%
正确样本个数:20,测试集样本总数:20
accuracy for class 1 and 3 : 100.00%
正确样本个数:20,测试集样本总数:20
accuracy for class 2 and 3 : 100.00%
(moshi) little1d@little1ddeMacBook-Pro Fisher % python main.py --dataset iris —method kfold --k 3
正确样本个数:33,测试集样本总数:33
accuracy for class 1 and 2 : 100.00%
正确样本个数:33,测试集样本总数:33
accuracy for class 1 and 3 : 100.00%
正确样本个数:34,测试集样本总数:34
accuracy for class 2 and 3 : 100.00%
```

针对 iris 数据库，可能逻辑稍微麻烦点，但是不难。首先针对分出来的三种实验划分训练集和数据集。

```
    if args.dataset == 'iris':
        X, y = load_data('iris')
        if args.method == 'random':
            X_train, X_test, y_train, y_test =
        elif args.method == 'kfold':
            X_train, X_test, y_train, y_test =
        elif args.method == 'leaveoneout':
            X_train, X_test, y_train, y_test =
        else:
             raise ValueError('Invalid method u
         # 由二维变到一维，在布尔索引的时候保证其为一组
         # IndexError: boolean index did not ma
         y_train = y_train.ravel()
         y_test = y_test.ravel()
         # 根据标签值将训练集和测试集分别划分为三个子集
         X_train_1 = X_train[y_train != 2]
         y_train_1 = y_train[y_train != 2]
         X_test_1 = X_test[y_test != 2]
         y_test_1 = y_test[y_test != 2]
         X_train_2 = X_train[y_train != 1]
         y_train_2 = y_train[y_train != 1]
         X_test_2 = X_test[y_test!= 1]
         y_test_2 = y_test[y_test!= 1]
         X_train_3 = X_train[y_train != 0]
         y_train_3 = y_train[y_train != 0]
         X_test_3 = X_test[y_test != 0]
         y_test_3 = y_test[y_test != 0]
```

可以看到需要先将 y_train 和 y_test 压缩到一维。从 load_data 函数返回的 shape 为(n,)，还是二维的矩阵，后面使用列表推导式时，写出布尔数组索引会报错。打印其结果如下：



```
y_train = y_train.ravel()
y_test = y_test.ravel()
X_train_1 = X_train[y_train != 2]
y_train_1 = y_train[y_train != 2]
X_test_1 = X_test[y_test != 2]
y_test_1 = y_test[y_test != 2]
X_train_2 = X_train[y_train != 1]
y_train_2 = y_train[y_train != 1]
X_test_2 = X_test[y_test!= 1]
y_test_2 = y_test[y_test!= 1]
X_train_3 = X_train[y_train != 0]
y_train_3 = y_train[y_train != 0]
X_test_3 = X_test[y_test != 0]
y_test_3 = y_test[y_test != 0]
y_test_1,y_test_2,y_test_3
```

```
(array([1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0,
        0, 0, 0, 0, 1, 1, 0, 1]),
 array([2, 2, 2, 2, 0, 2, 0, 0, 2, 2, 0, 2, 0, 0, 0, 0, 0, 2, 2, 0, 2, 2, 0,
        2, 0, 0, 0, 0, 0, 2, 2]),
 array([2, 1, 2, 1, 2, 2, 1, 1, 2, 2, 2, 2, 1, 1, 1, 2, 2, 1, 1, 1, 1, 2,
        2, 1, 2, 1, 1, 2, 2, 1]))
```

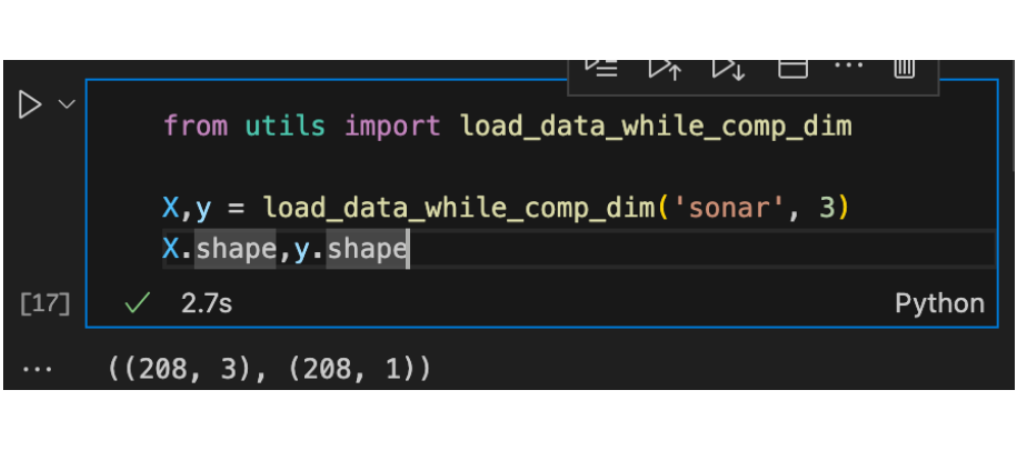
其实在这里还有一个非常重要的问题，就是 judge_sample 函数中只能返回 0，1。可是这里还有 2，还需要对每个实验进行一点点的转换。体现在 main 函数中就是改变样本标签，第一类改为 0，第二类改为 1.与 judge_sample 函数的设计保持一致。

针对不同维数绘制正确率折线图

有了前面的基础，针对不同维数绘制正确率折线，其实只需要修改数据集部分，选择不同的数据特征维数加载进 data 中进行训练和测试即可。

定义一个新的加载函数方法，可以切片特征维度。

```
def load_data_while_comp_dim(dataset, feature_n
    try:
        if dataset == 'sonar':
            data = fetch_ucirepo(id=151)
        elif dataset == 'iris':
            data = fetch_ucirepo(id=53)
    except Exception as e:
        raise ValueError(f"Error occurred while
    # 将 dataframes 转为 numpy arrays
     X = data.data.features.to_numpy()[:,0:feat
     y = data.data.targets.to_numpy()
     y_c = np.unique(y)
     for i in range(len(y_c)):
         y[y == y_c[i]] = i
     y = y.astype(int)
     return X, y
```



```
from utils import load_data_while_comp_dim

X,y = load_data_while_comp_dim('sonar', 3)
X.shape,y.shape
```

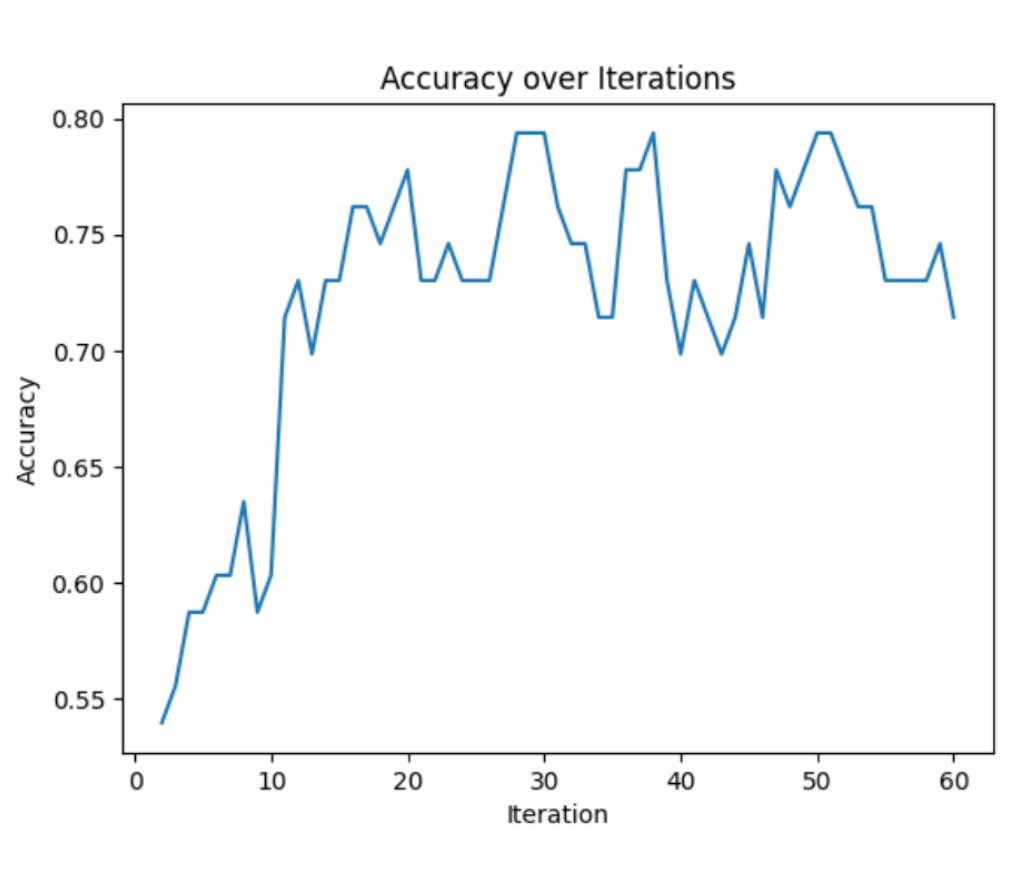
```
[17] ✓ 2.7s
((208, 3), (208, 1))
```

打印出来的形状也是正确的，只提取了前 3 种特征。

对 main 函数不需要过多修改，可以增加画图的逻辑。

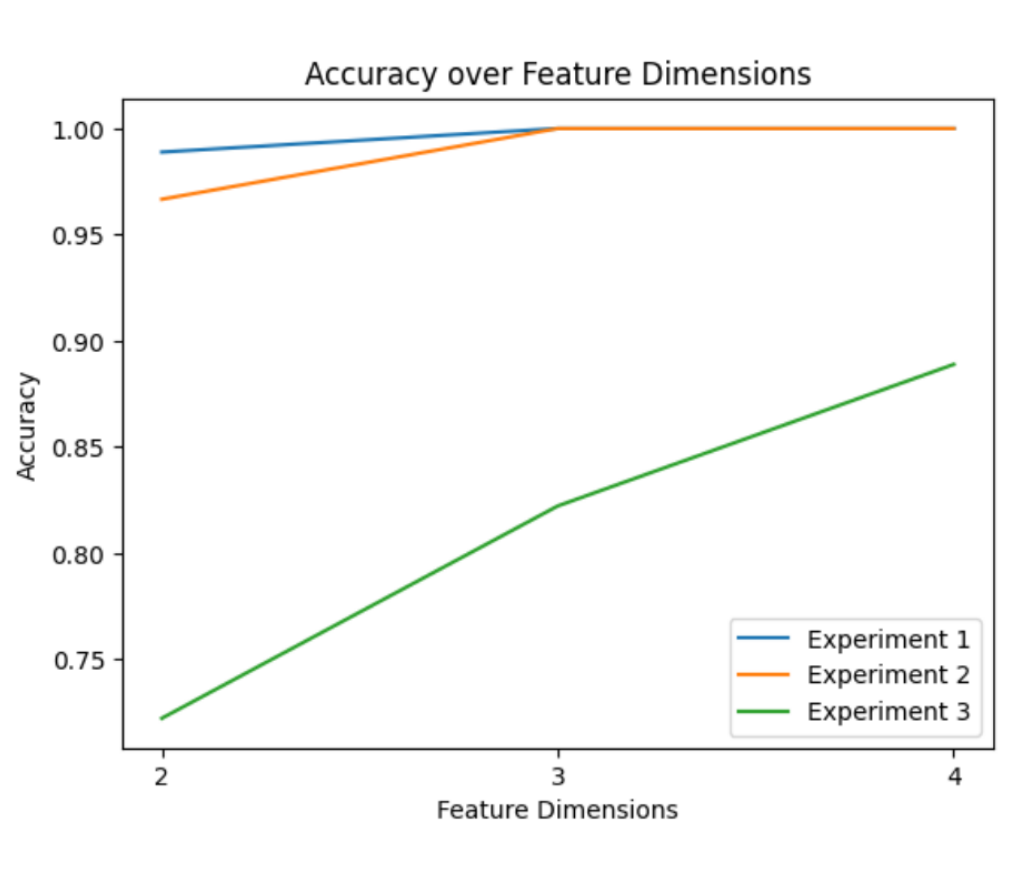
```
# sonar
from utils import *
accuracy_list = []
for i in range(2,61):
    X, y = load_data_while_comp_dim('sonar', i)
    X_train, X_test, y_train, y_test = random_s
    W ,u1, u2 = fisher_discriminant_analysis(X_
    y_pred = np.zeros(len(y_test))
    for i in range(len(y_test)):
        y_pred[i] = judge_sample(X_test[i,:],
     # print("len(y_test):", len(y_test))
     count=0
     for i in range(len(y_test)):
         if y_pred[i] == y_test[i]:
             count+=1
     # print(count)
     accuracy = count / len(y_test)
     accuracy_list.append(accuracy)
 accuracy_list
```

```
import matplotlib.pyplot as plt
plt.plot(range(2, 61), accuracy_list)
plt.title('Accuracy over Iterations')
plt.xlabel('Iteration')
plt.ylabel('Accuracy')
```



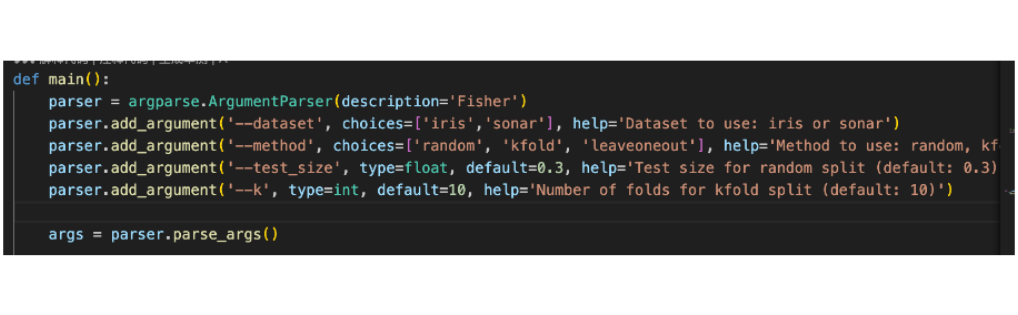
对于 iris 数据集，按照相同的逻辑进行计算并绘图。

```
# 绘图
feature = ['2','3','4']
for i in range(3):
    plt.plot(feature, [accuracy_list[i] for acc

plt.title('Accuracy over Feature Dimensions')
plt.xlabel('Feature Dimensions')
plt.ylabel('Accuracy')

 plt.legend()

 plt.show()
```



两类数据集上，随着特征维度数的增加，分类的正确率都是越高。对于 iris 数据库，仅仅使用 1 个特征正确率也非常高，这里的 Exp1、Exp2、Exp3 就是分别比较 1,2 类，1,3 类和 2,3 类。

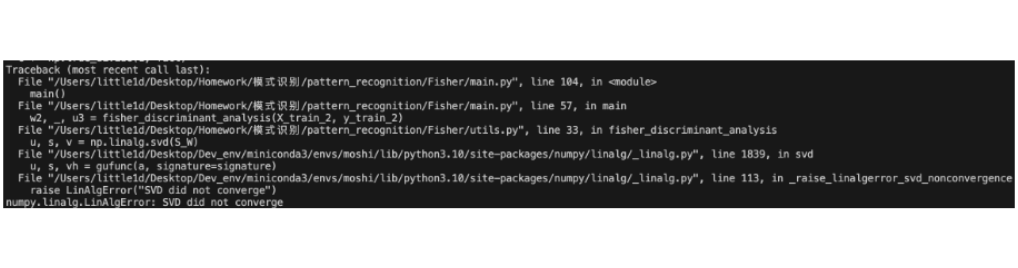
整体设计上，为了对两类数据集和三种数据集划分方法进行宏观的调控，我们可以定义一个 ArgumentParser 实例来管理所有参数。直接从命令行调用函数。



```
def main():
    parser = argparse.ArgumentParser(description='Fisher')
    parser.add_argument('--dataset', choices=['iris','sonar'], help='Dataset to use: iris or sonar')
    parser.add_argument('--method', choices=['random', 'kfold', 'leaveoneout'], help='Method to use: random, kf
    parser.add_argument('--test_size', type=float, default=0.3, help='Test size for random split (default: 0.3)
    parser.add_argument('--k', type=int, default=10, help='Number of folds for kfold split (default: 10)')

    args = parser.parse_args()
```

问题汇总与收获

1.



```
Traceback (most recent call last):
  File "/Users/little1d/Desktop/Homework/模式识别/pattern_recognition/Fisher/main.py", line 104, in <module>
    main()
  File "/Users/little1d/Desktop/Homework/模式识别/pattern_recognition/Fisher/main.py", line 57, in main
    w2, _, u3 = fisher_discriminant_analysis(X_train_2, y_train_2)
  File "/Users/little1d/Desktop/Homework/模式识别/pattern_recognition/Fisher/utils.py", line 33, in fisher_discriminant_analysis
    u, s, v = np.linalg.svd(S_W)
  File "/Users/little1d/Desktop/Dev_env/miniconda3/envs/moshi/lib/python3.10/site-packages/numpy/linalg/_linalg.py", line 1839, in svd
    u, s, vh = gufunc(a, signature=signature)
  File "/Users/little1d/Desktop/Dev_env/miniconda3/envs/moshi/lib/python3.10/site-packages/numpy/linalg/_linalg.py", line 113, in _raise_linalgerror_svd_nonconvergence
    raise LinAlgError("SVD did not converge")
numpy.linalg.LinAlgError: SVD did not converge
```

使用 numpy.linalg.svd 函数对 sonar 数据集进行奇异值分解时，算法没有收敛。可能是数据存在噪声。为了计算的简便，可以直接使用 np.linalg.inv()函数来计算矩阵的逆。

2.

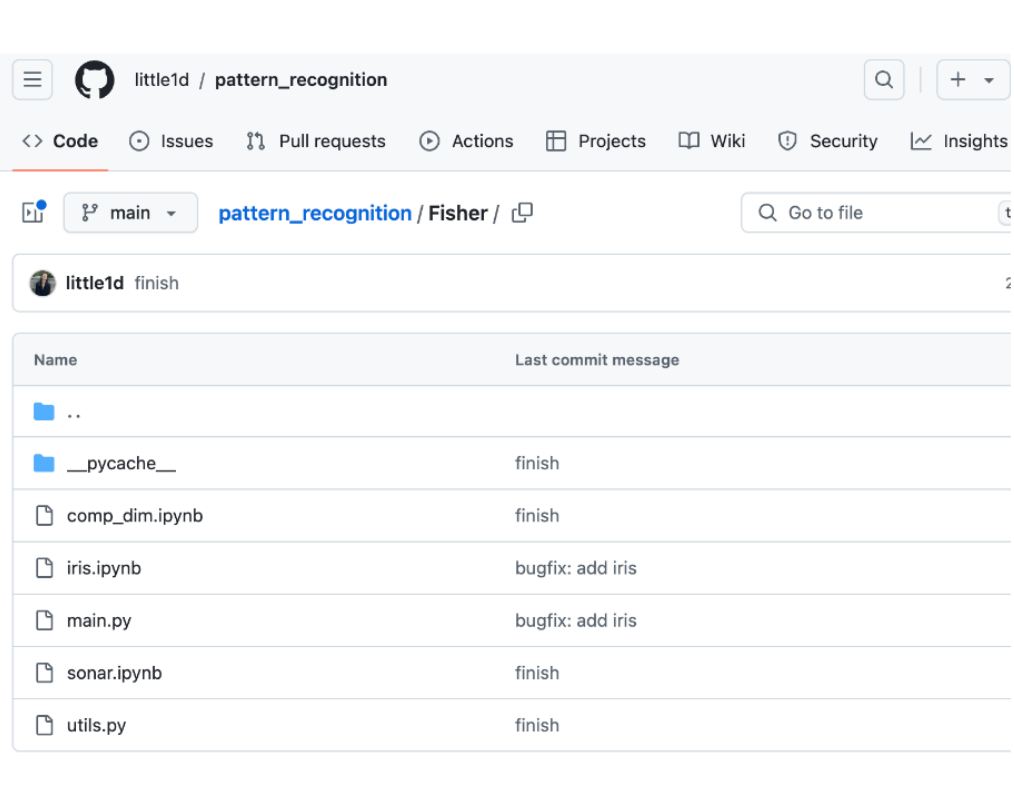
iris 数据集不支持 leaveoneout 方法，因为留一法仅仅只有 1 个样本，可能这个样本不在你此次的实验中。而且实验表明，在 iris 数据集上留一法并无太多参考价值。

3.

代码有很多重复的逻辑，可以考虑进一步的设计。

收获：

1. 熟悉使用了 numpy.ndarray，pandas.core.dataframe 数据结构，了解了数据形状、尺寸在代码运行中的重要性
2. 对 fisher 函数判别分析有了更深入的理解，必须要对算法了熟于心，coding 时候才能一气呵成。
3. 学会了使用 vscode、jupyter notebook 搭配进行高效 debug，欲速则不达，对于每一个函数做好测试，可以在后续的代码作业中写好单元测试
4. sklearn 库的使用，数据集的多种切分方法和参数



Name	Last commit message
..	
__pycache__	finish
comp_dim.ipynb	finish
iris.ipynb	bugfix: add iris
main.py	bugfix: add iris
sonar.ipynb	finish
utils.py	finish

https://github.com/little1d/pattern_recognition/tree/main/Fisher

目录架构：

1. comp_dim.ipynb：根据训练时不同特征维度训练并绘图
2. main.py：主入口
3. utils.py：封装了主体函数，包括数据加载，样本分类，fisher 判别分析等
4. sonar.ipynb/iris.ipynb：单元测试文件（写的很乱）

启动代码示例

```
python main.py --dataset sonar --method random
```

Bi-Directional Links　　Show